



How I Built an API for My RAG Chatbot



Natasha Ong

<https://www.linkedin.com/in/natasha-ong/>

```
127.0.0.1:8000/bedrock/query?text=What%20is%20NextWork
Pretty print ☒

{"detail": "Parameter validation failed:\nInvalid type for parameter\nretrieveAndGenerateConfiguration.knowledgeBaseConfiguration.knowledgeBaseId,\nvalue: None, type: <class 'NoneType'>, valid types: <class 'str'>\nInvalid type\nfor parameter\nretrieveAndGenerateConfiguration.knowledgeBaseConfiguration.modelArn, value:\nNone, type: <class 'NoneType'>, valid types: <class 'str'>"}
}
```



Introducing Today's Project!

In this project, we will demonstrate how to set up an API that connects a web app to our RAG chatbot set up with Amazon Bedrock. We are doing this project to learn how an API can make our chatbot accessible to other applications.

Tools and concepts

Services we used were Amazon Bedrock (Knowledge Base), Amazon S3, AWS IAM, Git. Key concepts we learnt include how APIs work, different API endpoints, environment variables, importing and installing dependencies using requirements.txt.

Project reflection

This project took us approximately 2.5 hours, including demo time. The most challenging part was troubleshooting errors with our API + resolving CLI command errors. It was most rewarding to run different endpoints to get different results.

We did this project today to really level up our understanding of RAG chatbots in AWS, and more importantly, how we can turn our chatbot into an accessible service that other applications can use. This project definitely met that goal and was fun!



Setting Up The Knowledge Base

To set up our Knowledge Base, we used S3 to store all the documents that we want our chatbot to know. The documents we uploaded contain information about a NextWork student's project documentation.

Amazon Bedrock is an AWS service that connects us to different AI models, and gives us tools that makes creating AI apps easier. We created a Knowledge Base in Bedrock to process the information in our S3 bucket, so our chatbot knows it allZ!

Our chatbot also needs access to two AI models to (1) embed our documents/queries (2) convert search results from the Knowledge Base into a chat response. We then synchronized our Knowledge Base with our S3 bucket to loads the documents in our Kbase.

✔ Knowledge Base 'nextwork-rag-documentation' created successfully. Sync one or more data sources to index your content for searching. Syncing can take from a few minutes to a few hours. ✕

[Go to data sources](#)

[Amazon Bedrock](#) > [Knowledge Bases](#) > nextwork-rag-documentation

nextwork-rag-documentation

[Test](#) [Delete](#)

Knowledge Base overview

Knowledge Base name
nextwork-rag-documentation

Knowledge Base description
This Knowledge Base stores all documentation at NextWork.

Service Role
[AmazonBedrockExecutionRoleForKnowledgeBase_cpgqe](#)

Knowledge Base ID
 GVQB1AQB4E

Status
 Available

Created date
February 21, 2025, 10:38 (UTC+13:00)

Log Deliveries
Configure log deliveries and event logs in the [Edit](#) page.

Retrieval-Augmented Generation (RAG) type
Vector store

[Edit](#)



Running CLI Commands Locally

When we ran a Bedrock command in the terminal, we initially got an error because our local terminal did not recognize AWS commands. To resolve this, we installed the AWS CLI (software for interacting with AWS resources using commands) locally.

We set up AWS access keys to give ourselves access to AWS and our account over the command line. This is like our way of logging into AWS (but over the terminal instead of the console).

To pass our access key credentials to the AWS CLI, we ran the command `aws configure` and had to provide our access key ID, Secret Access Key and default region name.

☰ IAM > Users > Nat-IAM-Admin > Create access key

✓ **Access key created**
This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can time.

Step 1
● Access key best practices & alternatives

Step 2 - optional
● Set description tag

Step 3
● **Retrieve access keys**

Retrieve access keys [Info](#)

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create new access key and make the old key inactive.

Access key	Secret access key
AKIAW3MEFRAF6KMMILJE	***** Show



Running Bedrock Commands

When we first ran a Bedrock command, we ran into an error because we needed to provide the values for our Knowledge Base's ID AND the AI Model's ModelARN.

To find the required values, we had to revisit the AWS Management Console to find the Knowledge Base ID and to find the AI model's ID (and ran another command to find the model ARN). The Bedrock command ran successfully and showed us a chat response!



Creating an API

An API is like a bridge between an application you're building and an external service (e.g. a chatbot built on Bedrock). Our API will make our chatbot accessible to other apps. This will be helpful for getting users an answer straight away.

Our API code has three main sections: (1) importing the libraries/dependencies, (2) loading environment variables from the local terminal, like `AWS_REGION` or `KNOWLEDGE_BASE_ID`, (3) defining the functions/features of the API in separate routes.

```
14 MODEL_ARN = os.getenv("MODEL_ARN")
15
16 app = FastAPI()
17
18 # Initialize Boto3 client for Bedrock Agent Runtime
19 def get_bedrock_client():
20     return boto3.client("bedrock-agent-runtime", region_name=AWS_REGION)
21
22 @app.get("/")
23 async def root():
24     return {"message": "Welcome to your RAG chatbot API!"}
25
26 @app.get("/bedrock/query")
27 async def query_bedrock(text: str = Query(..., description="Input text for the model")):
28     client = get_bedrock_client()
29     try:
30         response = client.retrieve_and_generate(
31             input={"text": text},
32             retrieveAndGenerateConfiguration={
33                 "knowledgeBaseConfiguration": {
34                     "knowledgeBaseId": KNOWLEDGE_BASE_ID,
35                     "modelArn": MODEL_ARN
36                 },
37                 "type": "KNOWLEDGE_BASE"
38             }
39         )
40         return {"response": response["output"]["text"]}
41     except Exception as e:
42         raise HTTPException(status_code=500, detail=str(e))
```



Installing Packages

To run the API, we had to install requirements like boto3, uvicorn, fastapi and dotenv. These packages are important because they help us with running the API and carries out tasks like loading the environment variables (boto3).

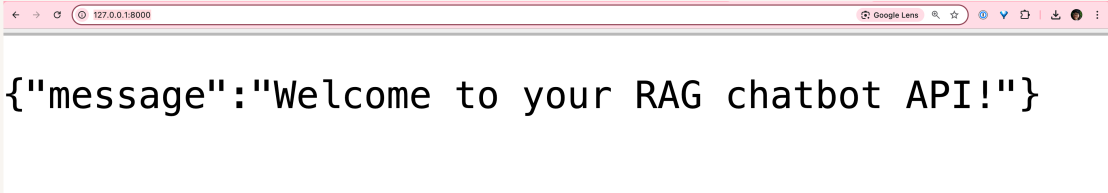
```
(venv) NextWork: $ pip3 list
```

Package	Version
-----	-----
annotated-types	0.7.0
anyio	4.8.0
boto3	1.36.20
botocore	1.36.25
click	8.1.8
fastapi	0.115.8
h11	0.14.0
idna	3.10
jmespath	1.0.1
pip	25.0
pydantic	2.10.6
pydantic_core	2.27.2
python-dateutil	2.9.0.post0
python-dotenv	1.0.1
s3transfer	0.11.2
six	1.17.0
sniffio	1.3.1



Testing the API

When we visited the root endpoint, we saw a message that says "Welcome to your RAG chatbot API!" This confirms that our API is working and our root is accessible. The message is exactly as we defined for our API's root endpoint in our main.py file.



```
{"message": "Welcome to your RAG chatbot API!"}
```



The Query Endpoint

The query endpoint connects an app with our chatbot. We tested the query endpoint by appending "bedrock/query?text=What%20is%20NextWork" to the end of our URL. The response was an error - parameter validation failed because we haven't defined KbaseID

Looking at the API code, we can see that the API calls for environment variables to protect information like your AWS access keys, knowledge base ID and modelARN and the AWS Region. To resolve the error in our query endpoint, we need to add env vars.

```
127.0.0.1:8000/bedrock/query?text=What%20is%20NextWork
Pretty print ☐

{"detail": "Parameter validation failed:\nInvalid type for parameter\nretrieveAndGenerateConfiguration.knowledgeBaseConfiguration.knowledgeBaseId,\nvalue: None, type: <class 'NoneType'>, valid types: <class 'str'>\nInvalid type\nfor parameter\nretrieveAndGenerateConfiguration.knowledgeBaseConfiguration.modelArn, value:\nNone, type: <class 'NoneType'>, valid types: <class 'str'>"}

```



Extending the API

In a project extension, we decided to extend our API by giving it the ability to let an application talk directly to the AI model. We need to update our API code by importing more modules, load a new environment variable and add a new endpoint defn.

We initially ran into an error with the new endpoint because we didn't add the MODEL_ID into our .env file (environment variable). Once we fixed it, we validated the new endpoint by asking our chatbot why the sky is blue - it responded perfectly!

When we use /bedrock/query, the chatbot uses the Knowledge Base to respond with specialized knowledge. But when we use /bedrock/invoke, the chatbot responds with general knowledge from the AI model (and does NOT use the Knowledge Base at all).

```
127.0.0.1:8000/bedrock/invoke?text=Why%20is%20the%20sky%20blue?
Pretty print ☐

{"response": "The sky appears blue because of a phenomenon called Rayleigh scattering, in which shorter (blue) wavelengths of light are scattered more than longer (red) wavelengths by the tiny molecules of gases in the Earth's atmosphere. This scattering effect gives the sky its blue color during the daytime, especially in the direction of the sun. As the sun rises or sets, the light has to travel through more of the atmosphere, which scatters the shorter wavelengths away from our line of sight, making the sun appear more red."}
```



NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

